

ARM PrimeCell Drivers

System Integration Guide

Primecell drivers system specification

© Copyright ARM Limited 2002. All rights reserved.

Proprietary notice

Words and logos marked with ® or ™ are registered trademarks or trademarks of ARM Limited in the EU and other countries, except as otherwise stated below in this proprietary notice. Other brands and names mentioned herein may be the trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties of merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- the document title
- the documents number
- the page number(s) to which your comments refer
- a concise explanation of your comments.

General suggestion for additions and improvements are also welcome.

Contents

1	ABOUT THIS DOCUMENT	9
1.1	Change control	9
1.1.1	Change history	9
2	SUMMARY	10
3	REFERENCES	10
4	DELIVERABLES	11
4.1	Key for deliverables diagrams	11
4.2	File Structure of deliverables	12
4.3	Drivers	12
4.4	Interrupt Controller	12
4.5	Software Framework	14
5	INSTALLATION	16
6	CODING CONVENTION	17
6.1	Introduction	17
6.2	Standard language extensions	17
6.3	System structure	17
6.4	Naming conventions	18
6.4.1	Project	18
6.4.2	Module	18
6.4.3	File Names	18
6.4.4	Variables	18
6.4.5	Macros	19
6.5	Access Functions	20
7	DRIVER DESIGN CRITERIA	21
7.1	Driver function	21
7.2	Callback functions	21
7.3	PrimeCell instances	22
7.4	PrimeCell variants	22
7.5	State data storage	22
7.5.1	Static state data	22
7.5.2	External state data	23
7.6	Module Initialization	23
7.7	Interrupt binding	24
8	HARDWARE INTEGRATION	25
8.1	Declaring peripheral instances	25

8.1.1	Adding a new instance of a peripheral	25
8.2	Specifying the base address of peripherals	26
8.2.1	Changing the base address of a peripheral	27
8.2.2	Adding the base address of an additional peripheral	27
8.3	Interrupt source format	28
8.4	Specifying the bit position of interrupt sources	28
8.4.1	Specifying the number of interrupt sources	28
8.4.2	Changing the bit position of an interrupt source	29
8.4.3	Adding the bit position of an additional interrupt source	29
8.5	Specifying a PrimeCell variant	30
8.6	Enabling timer functionality	30
8.6.1	Enabling timers	30
8.6.2	Disabling timers	30
9	INDIRECTION MODULE	31
9.1	Internal calls to the Interrupt Controller module	31
9.1.1	The delivered system	31
9.1.2	Interrupt Binding	31
9.1.3	Changing the software	31
9.2	Enabling and disabling interrupts to the processor core	32
9.2.1	The delivered system	32
9.2.2	Changing the software	32
9.3	Internal calls to a timer module	33
9.3.1	Timeout callbacks	33
9.3.2	Delay implementation	33
9.3.3	Changing the software	34
10	THE TEST MODULES	35
10.1	Test Functions	35
10.1.1	Constant definitions	35
10.1.2	Testing with a timeout handler	36
10.1.3	Changing the software	36
10.2	Bootstrap code	37
10.2.1	Stack and Heap	37
10.2.2	Changing the stack and heap settings	37
10.2.3	Cache/MMU/MPU support	38
10.2.4	Changing the cached core	38
10.2.5	Setting up memory management	39
10.2.6	Changing the memory configuration	40
10.2.7	Interrupt Controllers	41
10.2.8	Timers	41
10.3	Exception vectors	42
10.3.1	Reset vector	42
10.3.2	IRQ vector	42
10.3.3	FIQ vector	42
10.4	Test Application	43
10.4.1	The delivered system	43
10.4.2	Self-test modules	44
10.4.3	Changing the software	44

10.5	Specifying the modules to test	44
10.5.1	TEST_ALL_MODULES	44
10.5.2	TEST_MODULE	45
10.5.3	apOS_INSTANCE	46
10.5.4	TEST_NOINT	46
10.5.5	TEST_LIST	46
11	COMMON MODULE	47
11.1	Purpose of the Common module	47
11.2	Macros defining scope	47
11.2.1	PUBLIC	47
11.2.2	PRIVATE	47
11.2.3	PROTECTED	47
11.3	Further keywords	47
11.3.1	PACKED	47
11.3.2	INLINE	47
11.3.3	CONST	47
11.3.4	IRQ	47
11.4	Additional constants	48
11.4.1	TRUE	48
11.4.2	FALSE	48
11.4.3	aNULL	48
11.4.4	aRNULL	48
11.5	Additional macros	48
11.5.1	IGNORE	48
11.6	Primitive types	49
11.6.1	BOOL	49
11.6.2	BYTE8	49
11.6.3	UBYTE8	49
11.6.4	HWD16	49
11.6.5	UHWD16	49
11.6.6	WORD32	49
11.6.7	UWORD32	49
11.7	General function types	50
11.7.1	apTYPE_rCallback	50
11.8	Debug message macros	50
11.8.1	Constants for enabling debug	51
11.8.2	Constants for enabling buffered debug	52
11.8.3	Macros for debug output	53
11.9	Bitfield operation functions implemented as macros	54
11.9.1	General Items	55
11.9.2	Single bitfield macros	56
11.9.3	Dual bitfield macros	57
11.10	Other inline functions implemented as macros	58
11.10.1	MAX	58
11.10.2	MIN	58
11.10.3	apASSERT	58
11.11	Header Files for C and assembler	58
11.11.1	Translation header files	58
11.11.2	Header files affected	59

11.11.3	Preprocessor instructions	59
11.11.4	Comments	59
12	INTERRUPT CONTROLLER	60
12.1	Purpose of the Interrupt Controller module	60
12.2	Initializing the Interrupt Controller	60
12.3	Binding an interrupt source to a handler	62
12.4	Using the interrupt dispatcher	63
12.5	Configuring the Interrupt Controller	64
12.5.1	apINT_VERSION	64
12.5.2	apCONFIG_CORE_SUPPORTS	64
12.5.3	apOS_CONFIG_INT_REENTRANT	64
12.5.4	apOS_CONFIG_INT_USE_PREDISPATCH_CODE	65
12.5.5	apOS_CONFIG_INT_USE_POSTDISPATCH_CODE	65
12.5.6	apOS_CONFIG_INT_ISR_ISA	65
12.5.7	apOS_INT_CONFIG_SOURCES_USED	65
12.5.8	apOS_CONFIG_VIC_AT_0xFFFFF000	66
12.6	Developing on the Integrator Platform	66
12.7	Restrictions and features	66

Examples

Example 1 - File naming	18
Example 2 - Variable naming	19
Example 3 – Macro and enumerator naming	19
Example 4 – Access function name	20
Example 5 - PrimeCell driver function	22
Example 6 - General initialization function	23
Example 7 - Use of the initialization function	24
Example 8 - Format of enumerated types to declare peripheral instances	25
Example 9 - UART enumeration	25
Example 10 – Revised UART enumeration	26
Example 11 - Enumerators for specifying the base address of peripherals	26
Example 12 - Changing the base address of UART peripheral	27
Example 13 - Adding the base address of an additional UART	27
Example 14 - Enumerated type for interrupt bit positions	28
Example 15 - Specifying total number of interrupt sources	28
Example 16 - Changing the bit position of a timer interrupt source	29
Example 17 - Adding the bit position of an additional UART interrupt source	29
Example 18 - Selecting UART type PL011	30
Example 19 – Selecting a core type in ADS	38
Example 20 – Setting cache configuration for ARM720T	38
Example 21 – Example memory map	40
Example 22 - Self-test function	44
Example 23 - Determining modules to test	44
Example 24 – Incorrect manipulation of the TEST_ALL_MODULES macro	45
Example 25 - Specifying three interrupt sources	46
Example 26 – Single bitfield macro expansion	56
Example 27 – Dual bitfield macro expansion	57
Example 28 - Inclusion of common format header in C file	58
Example 29 - Inclusion of common format header in assembler file	59
Example 30 - Compiler control lines in the apintcfg.h file	59
Example 31 - Comments in the apintcfg.h file	59
Example 32 - Initializing an interrupt controller	60
Example 33 - Chained interrupt controllers	61
Example 34 - Binding and unbinding sources	62
Example 35 - Linking exception vectors	63

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A	12/02/01	First release of document
B	18/03/01	Updated details of interrupt controller configuration file
C	31/05/01	Added Integrator support (§12.6) and bootstrap (§10.2.1) sections
D	09/08/01	Modified interrupt configuration header
E	24/08/01	Updated with new details for bootstrap and memory map
F	19/06/02	Minor corrections to comply with Defect 135560
G	11/09/02	New release file structure

2 SUMMARY

This document describes how the PrimeCell drivers are integrated into an entire system. In particular, it covers:

- ♦ driver deliverables and installation
- ♦ the definition of system parameters
- ♦ the available library functions used by the drivers
- ♦ the interrupt controller
- ♦ the self-test routines and integration test application.

3 REFERENCES

	Document Number	Document Title
1	PL000-ISPC-1001	ARM PrimeCell driver framework: System Configuration API
2	PL000-ISPC-1002	ARM PrimeCell driver framework: System Indirection API
3	PL000-ISPC-1003	ARM PrimeCell driver framework: System Test API
4	PL000-ISPC-1004	ARM PrimeCell driver framework: Common Files API
5	PL001-ISPC-1001	ARM Interrupt Controller API
6	PLnnn-ISPC-1000	ARM PrimeCell driver API for PrimeCell PLnnn
7	PLnnn-TREP-1000	ARM PrimeCell driver test report for PrimeCell PLnnn

4 DELIVERABLES

4.1 Key for deliverables diagrams

The diagrams in §4.3, §4.4 and §4.5 use the key below to indicate file types.

	Directory
	Private header file, used within the PrimeCell driver build
	Public header file, included by applications wishing to access PrimeCell driver functionality
	C source code
	Assembler source code
	Object code
	Documentation
	Project file specifying build details
	Archived or compressed file
	Text file

4.2 File Structure of deliverables

Generic PrimeCell software is delivered as one directory, PrimeCell_gen_sw

Figure 4-1 illustrates the directory structure for the software

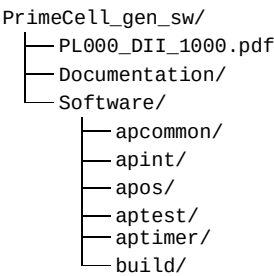


Figure 4-1 PrimeCell Generic Software Directory Structure

4.3 Drivers

Each driver is supplied with the deliverables detailed in Figure 4-2. The module name replaces the generic term *mod* and the PrimeCell number replaces *nnn* below. For example, for a UART, the directory is *apuart* and the API document is *PL010- ISPC- 1000 .pdf*.

In some cases, there might be further files provided for support purposes. For example, the Vectored Interrupt Controller (PL190) includes an interrupt dispatcher file *vicdispatch.s*.

Figure 4-2 - Driver deliverables (packed in directories PL*nnn*_*)

	<i>apmod</i>	The module code directory. This must be added at the base of the development tree in §4.1.
	<i>apmod.h</i>	Public header file for driver module
	<i>mod.h</i>	Private header file for driver module
	<i>mod.c</i>	Source code for driver module
	<i>modtst.c</i>	Self-test module for driver module (see §10.4.2).
	PL <i>nnn</i> - ISPC- 1000 .pdf	ARM PrimeCell driver API specification
	PL <i>nnn</i> - TREP- 1000 .pdf	ARM PrimeCell driver test report

4.4 Interrupt Controller

Software to control the Micropack Interrupt Controller is released as object code as shown in Figure 4-3.

Figure 4-3 – Interrupt Controller deliverables (packed in directory Software/)

	<i>apint</i>	The interrupt controller, object code release for testing (see §12)
---	--------------	---

	<code>dispatch.o</code>	Object code for interrupt dispatcher
	<code>apint.h</code>	Public header file for interrupt controller
	<code>int.h</code>	Private header file for interrupt controller (not used in object release)
	<code>int.o</code>	Object code for interrupt controller
	<code>inttst.c</code>	Self-test module for interrupt controller (see §10.4.2).

4.5 Software Framework

The release of a driver is accompanied with the deliverables shown in Figure 4-4.

Figure 4-4 - Software Framework deliverables (packed in directory Software/)

	apcommon	The common module, a selection of generic header files(see §11)
	apbitops.h	Header file defining bitfield operations (see §11.9)
	aptypes.h	Header file defining (see §11)
	asmacros.h	Header file allowing common assembler/C code usage (see §11.11.1)
	cmacros.h	Header file allowing common assembler/C code usage (see §11.11.1)
	apos	The indirection module (see §8.5)
	apos.h	Public header file for indirection module
	applatfm.h	Hardware system specification header (see §5)
	apintcfg.h	Configuration file for interrupt controller
	os.c	C source code for indirection module
	oss.s	Assembler source code for indirection module
	aptest	Test application functionality and bootstrap code (see §10)
	aptest.h	Public header file for test functions (see §10.1)
	boot.s	Source code for minimal bootstrap for test application (see §10.2.7)
	main.c	Main application function for test application (see §10.2.7)
	mmu_mpu.s	Configuration data for defining memory map for MMU or MPU (§10.2.5)
	pagetab.s	Blank space for page table if required for a core with MMU
	sysdefs.h	Private header file specifying stack and heap details (used by bootstrap)
	test.c	Source code for test functions (see §10.1)
	test.h	Private header file for test functions (see §10.1)
	vectors.s	Minimal exception vectors for test application (see §10.2.7)
	build	Directory with sample projects for test application (see §10.2.7)
	ads1.1	Sample project directory for ADS1.1
	primecell.mcp	ADS project file for test application (see §10.2.7)
	sdt2.51	Sample project directory for SDT2.51
	primecell.apj	SDT project file for test application (see §10.2.7)
	PL000-ISP-1001.pdf	ARM PrimeCell drivers system configuration API specification (see §5)

	PL000-ISPC-1002.pdf	ARM PrimeCell drivers system indirection API specification (see §8.5)
	PL000-ISPC-1003.pdf	ARM PrimeCell drivers test module API specification (see §10)
	PL000-ISPC-1004.pdf	ARM PrimeCell drivers common module API specification (see §11)

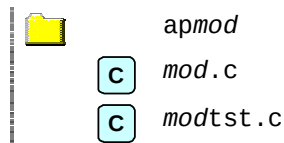
5 INSTALLATION

To install the drivers, follow these steps:

- ◆ Unzip all supplied files into a common root directory using a standard tool (such as *tar* under UNIX or WinZip under Windows). Subdirectories are created during the unzipping procedure.
- ◆ Locate the project file for ADS, named `primecell.mcp`, or for SDT, named `primecell.apj`.

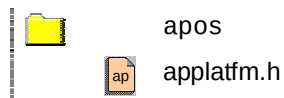


- ◆ Open the project file – this project builds the test application (see §10.4.1). It already contains the framework for driver development as below:
 - bootstrap code.
 - test application (see §10.2.7)
 - test functions (see §10.1)
 - indirection module (see §8.5)
 - interrupt controller as object code (see §12).
- ◆ For each released driver module *MOD*:
 - Add the source code files in the top-level directory *apmod* to the project



For example, to include the UART module, add `apuart/uart.c` and `apuart/uarttst.c` to the project.

- ◆ Locate the system configuration file `applatfm.h`.



- ◆ Specify the system layout by adding the relevant values into `applatfm.h` (see §8). The required items are:
 - create an enumeration of the peripherals present for each driver type (see §8.1 and Example 8)
 - specify the PrimeCell variant to be supported, if the driver supports multiple variants (see §8.5 and Example 18)
 - specify the base address of each peripheral (see §8.2 and Example 11)
 - specify the interrupt source(s) for each peripheral (see §8.4 and Example 14).
- ◆ Specify the peripherals to be tested by the application by setting the macro `TEST_ALL_MODULES` defined in `applatfm.h` (see §10.5 and Example 23).
- ◆ Build and run the project. This runs the self-test modules for each peripheral. Each reports the tests performed and their results, concluding with a pass or fail for that peripheral. The application concludes with a pass or fail result for the overall system test.

6 CODING CONVENTION

6.1 Introduction

The PrimeCell drivers adhere to a coding standard. This presents the following benefits:

- ◆ consistency of code layout and structure
- ◆ standardisation of naming of code elements
- ◆ application of *good practice* in production of code.

The coding standard used is documented in ARM document **SSB00 WKIN 0001**.

6.2 Standard language extensions

Some additional keywords, constants and types are defined for use within the company. The adoption of standard forms of these common items is to eliminate ambiguity. These include the extensions described in:

- ◆ Primitive types
- ◆ Macros defining scope
- ◆ Further keywords
- ◆ Additional constants
- ◆ Additional macros.

These extensions are described in section 11.

6.3 System structure

Systems are composed of modules; modules are made up of routines and associated data. In general, a module consists of a single source code file and any associated headers, a routine corresponds to a single C or Assembler procedure.

A module must consist of three parts:

- ◆ The *public* header contains all variables and functions that are publicly exported and are therefore externally visible.
- ◆ The *private* header contains all other definitions, of module scope and internal to the module. Multiple private header files can be used where this is an aid to clarity.
- ◆ The source file contains the code implementation.

6.4 Naming conventions

6.4.1 Project

All PrimeCell drivers use the project code *ap* (for *ARM PrimeCell*). This is prepended to the start of all items of public scope relating to the project.

6.4.2 Module

Each PrimeCell driver is assigned a module code of three to five alphabetic, uppercase characters, identifying the driver type. For example, the UART driver has module code *UART*.

6.4.3 File Names

Filenames must not exceed the DOS-style limit of eight characters for the name and three for the extension. An example is shown in Example 1.

- ◆ The primary source code file for each module is a lower case version of the module code, with the appropriate extension (.c) appended.
- ◆ The private header file for a module name are named with a lower case version of the module code, with the appropriate extension (.h) appended. This header is used internally by the source code.
- ◆ The public header file for a module name is named with a lower case version of the module code, preceded by the project code (ap) with the appropriate extension (.h) appended.

Example 1 - File naming

uart.c	(source code)
uart.h	(private header)
apuart.h	(public header)

6.4.4 Variables

All variables adhere to the following naming conventions:

- ◆ public variable names are preceded with the project code (ap)
- ◆ variables with module scope or higher (globals) start with the module code, followed by an underscore (for example *UART_*)
- ◆ variable names are lower case with the initial letter of the name and of any words within the name capitalised.

A single character *tag* can precede the variable name. This can be one of:

r	Pointer to a routine
p	Pointer to any item but a routine
o	Opaque type (definition of type encapsulated in another module)
e	Enumerated type definition or variable of enumerated type
s	Variable or type which is a structure or union
t	Definition of other types (including definition of arrays)
x	Unused item in enum or struct type definition
k	Constant

Some example variable names are shown in Example 2.

Example 2 - Variable naming

UWORD32 SomeVariable;	A local variable of unsigned word type
UWORD32 *pSomePointer;	A local variable of pointer type
typedef UWORD32 tWordType;	A type definition of local scope
UWORD32 apUART_SomeVariable;	A public global variable of unsigned word type
UWORD32 * UART_pSomePointer;	A private global variable of pointer type
typedef UWORD32 apUART_tWordType;	A public global type definition

6.4.5 Macros

All macros, pre-processor constants and enumerators adhere to the following naming conventions.

An example of a public enumeration is shown in Example 3.

- ◆ public macro names are preceded with the project code (ap)
- ◆ macros with module scope or higher (globals) start with the module code, followed by an underscore (for example, UART_)
- ◆ macro names are in upper case with any words delimited by underscores.

Example 3 – Macro and enumerator naming

```
typedef enum apUART_xStopBits
{
    apUART_ONE_STOP_BIT,    // One stop bit.
    apUART_TWO_STOP_BITS    // Two stop bits.
}
apUART_eStopBits;
```

6.5 Access Functions

The naming of access functions has been standardised to some degree. The function name consists of several words with initial letters capitalised. The last word is the function applied. The name can be broken down as in Example 4.

Example 4 – Access function name

apUART_BaudRateGet	
ap	Project name
UART	Module name
BaudRate	Item accessed
Get	Data access function

The defined access function names are:

Disable	Deactivate a functional block
Enable	Activate a functional block
Get	Retrieve state information from peripheral
Inform	Pass data for storage in a driver without affecting underlying peripheral state
Read	Retrieve data stored within peripheral
Receive	Receive data from device attached to peripheral. The data flows through the module interface.
Set	Pass state information to peripheral
Transmit	Send data to device attached to peripheral. The data flows through the module interface.
Write	Store data within peripheral

7 DRIVER DESIGN CRITERIA

7.1 Driver function

The PrimeCell drivers are intended to implement a physical layer driver, directly accessing the PrimeCell registers. They are not intended for porting to other register maps.

The drivers are extensible and it is desirable in some cases to add the higher layers of a protocol stack on top of the supplied driver.

Although the driver model varies according to the device being driven, the general model for a driver supporting data flow is that the driver provides interrupt-driven data communication with callback. Polled communication is not usually supported.

The general format of usage is:

- ◆ initialization of the driver, specifying the device to be driven (see §7.6)
- ◆ specifying parameters for setup. For example a communications device can specify data transfer speed
- ◆ initiating a data transfer. This is typically achieved by supplying the following:
 - a data buffer for transmission or an empty buffer for reception
 - a number of data elements to be transferred
 - a callback function (see §7.2) to be called on completion.
- ◆ the driver *Interrupt Service Routine* (ISR) is then responsible for transferring the data under interrupt control
- ◆ after the transfer is complete, the driver calls the passed callback function to report the success or failure of the transfer.

7.2 Callback functions

Callback functions are supplied by the code invoking the driver to allow that code to be notified of the completion of asynchronous functions (such as data transfer) requested from the driver.

«Note» Callback functions are called from within an ISR in *Interrupt (INT) ()* mode.

Care must be taken in the design of such functions, noting especially the points below.

- ◆ the user of the driver is responsible for ensuring that callback functions are of minimal length and complexity and suitable for use within an interrupt handler
- ◆ any global data accessed by the callback function and the main code (typically a flag set in the callback and polled in the main code) must be declared ***volatile***.
- ◆ where an OS is being used, OS functions must be layered between the driver callback and the user process as required.

7.3 PrimeCell instances

Each PrimeCell driver controls one or more PrimeCell peripherals of a common type.

For example, a single UART driver can control one or more PrimeCell PL010 UARTs.

Each peripheral controlled is termed an *instance* of the PrimeCell.

Each function call to a PrimeCell driver specifies the instance of the PrimeCell to be addressed. This is specified by the first parameter to the call as in Example 5. This parameter is an opaque type `apOS_MOD_oId`, where *MOD* is replaced by the driver name in upper case, and the interpretation of the parameter is described in §7.4.

Example 5 - PrimeCell driver function

```
PUBLIC void apUART_Enable(apOS_UART_oId Id);
```

There is no specific limit to the number of PrimeCell instances that can be controlled by one driver.

7.4 PrimeCell variants

Where multiple variants of a PrimeCell exist (for example the PL010 and PL011 UARTs), the drivers are designed to support any non-obsolete variants by means of a compile-time switch (see §8.5).

«Note» Because register maps and hardware behaviour varies between variants, each driver must be built to support only one PrimeCell variant. Multiple variants must not be mixed within one system.

7.5 State data storage

Data is required to maintain state for each PrimeCell instance controlled by the driver.

The required size of the state data is specified in the API document for each driver.

The state data can be stored in one of two forms as described below.

7.5.1 Static state data

In this mode, the necessary RAM space for each PrimeCell instance controlled by the driver is specified at compile time and reserved as static variable space.

The number of instances available for a driver *MOD* is defined by a constant `apOS_MOD_MAXIMUM`.

Each instance is identified by an integer index, with `apOS_MOD_oId` implemented as an enumeration (see §8.1 and Example 9).

This is the default setting for the driver.

7.5.2 External state data

In this mode, the driver reserves no static variable space at compile time. Instead, the space is reserved externally to the driver.

This option is selected by defining `apOS_NO_STATIC_STATE` as `TRUE` in `aplatfm.h` (see §5)

The number of instances available for a driver is not limited by that driver.

A pointer to a block of state data identifies each instance for the driver, with `apOS_MOD_oId` implemented as a pointer.

The size of the state data block can be found by invoking the function `apMOD_StateSizeGet`.

7.6 Module Initialization

A standard initialization call is provided for each peripheral driver. Its general format is shown in Example 6, where *MOD* is replaced by the driver name in upper case.

Example 6 - General initialization function

```
PUBLIC void apMOD_Initialize (apOS_MOD_oId oId,  
                             apOS_System_eBaseAddress eBase,  
                             UWORD32 Interrupts,  
                             CONST apOS_INT_oInterruptSource *pSources,  
                             CONST apMOD_sInitialData *pInitial);
```

The usage of the parameters is as follows.

Parameter	Purpose
<code>oId</code>	Identification of the peripheral instance to be initialized (see §7.4)
<code>eBase</code>	Base address of the peripheral (see §8.2)
<code>Interrupts</code>	Number of interrupt sources. The driver binds this many interrupt sources to an interrupt handler routine and enables the sources (see §8.4.1)
<code>pSources</code>	Pointer to a list of interrupt sources (see §8.4)
<code>pInitial</code>	Pointer to peripheral specific initialization data structure. This is driver-specific. Refer to the driver API for details.

Before use, each PrimeCell instance must be initialized by a call to the initialization function.

Using any driver call for a PrimeCell instance before calling the initialization function for that instance leads to unpredictable results.

An example of the use of the initialization function is shown in Example 7, which initializes a PrimeCell MCI peripheral with a 48MHz reference clock speed.

Example 7 - Use of the initialization function

```
{
    apMCI_sInitialData sSetupData;
    apOS_INT_oInterruptSource MyInterrupts[]={apOS_INT_MCI0, apOS_INT_MCI1};
    apOS_System_eBaseAddress RegBase= apOS_SYSTEM_BASE_MCI

    sSetupData.DeviceClock=48000000UL;          /*set 48MHz device clock*/
    apMCI_Initialize(oId, RegBase,2,MyInterrupts,& sSetupData);
}
```

7.7 Interrupt binding

On initialization, the driver binds (associates) the specified interrupt sources with the driver interrupt service routine.

This binding uses the indirected interrupt controller functions (see §9.1) to access the interrupt controller (see §12.3), although a different interrupt controller might be used in a final system.

If this mechanism is not desired, the initialization function might be passed zero as the number of interrupts.

8 HARDWARE INTEGRATION

This chapter describes the changes and additions you must make to the supplied software if you make changes or additions to the default hardware supplied [1].

8.1 Declaring peripheral instances

For each peripheral that has multiple instances, there is an enumerated type that:

- ◆ declares each instance of the peripheral
- ◆ declares the total number of available instances of the peripheral.

Example 8 shows the general format of these enumerated types for a generalised driver *MOD*.

Example 8 - Format of enumerated types to declare peripheral instances

```
typedef enum apOS_MOD_xId
{
    apOS_MOD_0 = 0,      // First instance of peripheral
    apOS_MOD_1 = 1,      // Second instance of peripheral
    ...                  // Further instances of peripheral
    apOS_MOD_MAXIMUM     // Number of available peripherals
}
apOS_MOD_oId;
```

In Example 8, the enumerator `apOS_MOD_MAXIMUM` does not have a value explicitly assigned to it. Its value is therefore that of the previous enumerator, incremented by one.

This syntax ensures that its value is always correct, if the peripheral instances are always numbered consecutively from 0.

8.1.1 Adding a new instance of a peripheral

1. Add a new enumerator declaring the new instance of the peripheral.
2. Ensure that the enumerator, `apOS_MOD_MAXIMUM`, giving the number of available instances remains correct.

The enumerated type listing the UARTs is `apOS_UART_oId`. Its declaration in the example header file is shown in Example 9.

Example 9 - UART enumeration

```
typedef enum apOS_UART_xId
{
    apOS_UART_0 = 0,      // Specifies UART 0
    apOS_UART_1 = 1,      // Specifies UART 1
    apOS_UART_MAXIMUM     // Number of UARTs in the system
}
apOS_UART_oId;           //Enumeration of UARTs available for the system
```

To declare the extra UART, you add an enumerator `apOS_UART_2`, as shown in Example 10.

Example 10 – Revised UART enumeration

```
typedef enum apOS_UART_xId
{
    apOS_UART_0 = 0,           // Specifies UART 0
    apOS_UART_1 = 1,           // Specifies UART 1
    apOS_MY_UART = 2,          // Specifies new UART added to system
    apOS_UART_MAXIMUM           // Number of UARTs in the system
}
apOS_UART_oId;                //Enumeration of UARTs available for the system
```

The value of the enumerator `apOS_UART_MAXIMUM` is now implicitly set to 3. It is still giving the correct number of UARTs in the system, and so no more editing is necessary.

8.2 Specifying the base address of peripherals

The enumerated type `apOS_System_eBaseAddress` specifies the base address of various peripherals. The value of each enumerator is the address, in bytes.

The enumerators for this type are defined in the file `applatfm.h`, and the shipped values are shown in Example 11

Example 11 - Enumerators for specifying the base address of peripherals

```
typedef enum apOS_System_eBaseAddress
{
    apOS_SYSTEM_BASE_TIMER0      = (WORD32) 0x13000000, // Timer 0
    apOS_SYSTEM_BASE_TIMER1      = (WORD32) 0x13000100, // Timer 1
    apOS_SYSTEM_BASE_TIMER2      = (WORD32) 0x13000200, // Timer 2
    apOS_SYSTEM_BASE_INT         = (WORD32) 0x14000000, // Interrupt controller
    apOS_SYSTEM_BASE_RTC         = (WORD32) 0x15000000, // Real Time Clock
    apOS_SYSTEM_BASE_UART0       = (WORD32) 0x16000000, // UART 0
    apOS_SYSTEM_BASE_UART1       = (WORD32) 0x17000000, // UART 1
    apOS_SYSTEM_BASE_KMI0        = (WORD32) 0x18000000, // Keyboard/Mouse Interface 0
    apOS_SYSTEM_BASE_KMI1        = (WORD32) 0x19000000, // Keyboard/Mouse Interface 1
    apOS_SYSTEM_BASE_GPIO        = (WORD32) 0x1B000000  // General Purpose IO
}
apOS_System_eBaseAddress;
```

8.2.1 Changing the base address of a peripheral

If you change the base address of a peripheral, you must change the value of the corresponding enumerator.

For example, if you change the base address of one UART to 0x72000000, you must change the value of ap0S_SYSTEM_BASE_UART0, as shown in Example 12.

Example 12 - Changing the base address of UART peripheral

```
typedef enum ap0S_System_eBaseAddress
{
    ap0S_SYSTEM_BASE_TIMER0      = (WORD32) 0x13000000, // Timer 0
    ap0S_SYSTEM_BASE_TIMER1      = (WORD32) 0x13000100, // Timer 1
    ap0S_SYSTEM_BASE_TIMER2      = (WORD32) 0x13000200, // Timer 2
    ap0S_SYSTEM_BASE_INT         = (WORD32) 0x14000000, // Interrupt controller
    ap0S_SYSTEM_BASE_RTC         = (WORD32) 0x15000000, // Real Time Clock
    ap0S_SYSTEM_BASE_UART0       = (WORD32) 0x72000000, // UART 0 at new address
    ap0S_SYSTEM_BASE_UART1       = (WORD32) 0x17000000, // UART 1
    ap0S_SYSTEM_BASE_KMI0        = (WORD32) 0x18000000, // Keyboard/Mouse Interface 0
    ap0S_SYSTEM_BASE_KMI1        = (WORD32) 0x19000000, // Keyboard/Mouse Interface 1
    ap0S_SYSTEM_BASE_GPIO        = (WORD32) 0x1B000000 // General Purpose IO
}
ap0S_System_eBaseAddress;
```

8.2.2 Adding the base address of an additional peripheral

If you add a new peripheral, you must add a corresponding enumerator to specify its base address. The enumerator must be named ap0S_SYSTEM_BASE_PERIPHERAL, where PERIPHERAL is the name of the peripheral, in uppercase.

For example, if you add an extra UART, you must add an enumerator to specify its base address, as shown in Example 13.

Example 13 - Adding the base address of an additional UART

```
typedef enum ap0S_System_eBaseAddress
{
    ap0S_SYSTEM_BASE_TIMER0      = (WORD32) 0x13000000, // Timer 0
    ap0S_SYSTEM_BASE_TIMER1      = (WORD32) 0x13000100, // Timer 1
    ap0S_SYSTEM_BASE_TIMER2      = (WORD32) 0x13000200, // Timer 2
    ap0S_SYSTEM_BASE_INT         = (WORD32) 0x14000000, // Interrupt controller
    ap0S_SYSTEM_BASE_RTC         = (WORD32) 0x15000000, // Real Time Clock
    ap0S_SYSTEM_BASE_UART0       = (WORD32) 0x16000000, // UART 0
    ap0S_SYSTEM_BASE_UART1       = (WORD32) 0x17000000, // UART 1
    ap0S_SYSTEM_BASE_MY_UART     = (WORD32) 0x72000000, // UART 2 added
    ap0S_SYSTEM_BASE_KMI0        = (WORD32) 0x18000000, // Keyboard/Mouse Interface 0
    ap0S_SYSTEM_BASE_KMI1        = (WORD32) 0x19000000, // Keyboard/Mouse Interface 1
    ap0S_SYSTEM_BASE_GPIO        = (WORD32) 0x1B000000 // General Purpose IO
}
ap0S_System_eBaseAddress;
```

8.3 Interrupt source format

The interrupt source format is opaque to the driver modules. The source is specified by a single word that is passed to the interrupt controller. The interpretation of this word depends on the design of the interrupt controller.

The format used by the supplied interrupt controller is:

- ♦ its[8:15] specify the controller to be used as an enumeration value (see §8.1)
- ♦ bits[0:7] specify the interrupt source to be passed to that interrupt controller.

8.4 Specifying the bit position of interrupt sources

The enumerated type `apOS_INT_oInterruptSource` specifies the bit position of each interrupt source.

The enumerators for this type are defined in the file `apPlatform.h`, and a section of the shipped values are shown in Example 14. Here the interrupt controller is defined as a constant in bits 8-15 as below.

Example 14 - Enumerated type for interrupt bit positions

```
#define apOS_INTCTL_AP (0<<8)           //interrupt controller for Integrator AP
#define apOS_INTCTL_LM (1<<8)           //interrupt controller for Integrator LM

typedef enum apOS_INT_xInterruptSource
{
    apOS_INT_PROGRAMMED = apOS_INTCTL_AP | 0x00, // Software interrupt source
    apOS_INT_UART0      = apOS_INTCTL_AP | 0x01, // UART 0 interrupt source
    apOS_INT_UART1      = apOS_INTCTL_AP | 0x02, // UART 1 interrupt source
    apOS_INT_KMI0       = apOS_INTCTL_AP | 0x03, // Keyboard/Mouse interrupt source
    apOS_INT_KMI1       = apOS_INTCTL_AP | 0x04, // Keyboard/Mouse interrupt source
    .
    .
    .
    apOS_INT_NONE       = 0x00000020, // No source - for testing only
    apOS_INT_32BIT      = 0x7FFFFFFF  // Forces compiler to use 32-bit int for enum
}
apOS_INT_oInterruptSource;
```

8.4.1 Specifying the number of interrupt sources

Each interrupt controller can control up to 32 interrupt sources. By default, enough space is assigned for this many sources. However, if a lesser number of sources are required, this can be specified by defining the constant `apOS_INT_TOTAL_SOURCES_USED`. This constant must specify the total number of interrupt sources used across all interrupt controllers as in Example 15.

Example 15 - Specifying total number of interrupt sources

```
#define apOS_INT_TOTAL_SOURCES_USED 16
```

8.4.2 Changing the bit position of an interrupt source

If you change the bit position of an interrupt source, you must change the value of the corresponding enumerator.

For example, if you change the interrupt source for UART 0 to use bit 17, you must change the value of `apOS_INT_UART0` to also be 17, as shown in Example 16.

Example 16 - Changing the bit position of a timer interrupt source

```
typedef enum apOS_INT_xInterruptSource
{
    apOS_INT_PROGRAMMED = apOS_INTCTL_AP | 0x00, // Software interrupt source
    apOS_INT_UART0      = apOS_INTCTL_AP | 0x11, // Modified UART 0 interrupt source
    apOS_INT_UART1      = apOS_INTCTL_AP | 0x02, // UART 1 interrupt source
    apOS_INT_KMI0       = apOS_INTCTL_AP | 0x03, // Keyboard/Mouse interrupt source
    apOS_INT_KMI1       = apOS_INTCTL_AP | 0x04, // Keyboard/Mouse interrupt source
    .
    .
    apOS_INT_NONE       = 0x00000020, // No source - for testing only
    apOS_INT_32BIT      = 0x7FFFFFFF // Forces compiler to use 32-bit int for enum
}
apOS_INT_oInterruptSource;
```

8.4.3 Adding the bit position of an additional interrupt source

If you add an additional interrupt source, for example by adding a new peripheral, you must add a corresponding enumerator. The enumerator must be named `apOS_INT_PERIPHERAL`, where `PERIPHERAL` is the name of the peripheral, in uppercase.

For example, if you add an extra UART, you must add an enumerator named `apOS_INT_MY_UART` to specify the bit position of its interrupt source, as shown in Example 17.

Example 17 - Adding the bit position of an additional UART interrupt source

```
typedef enum apOS_INT_xInterruptSource
{
    apOS_INT_PROGRAMMED = apOS_INTCTL_AP | 0x00, // Software interrupt source
    apOS_INT_UART0      = apOS_INTCTL_AP | 0x01, // UART 0 interrupt source
    apOS_INT_UART1      = apOS_INTCTL_AP | 0x02, // UART 1 interrupt source
    apOS_INT_MY_UART    = apOS_INTCTL_AP | 0x11, // Added my UART interrupt source
    apOS_INT_KMI0       = apOS_INTCTL_AP | 0x03, // Keyboard/Mouse interrupt source
    apOS_INT_KMI1       = apOS_INTCTL_AP | 0x04, // Keyboard/Mouse interrupt source
    .
    .
    apOS_INT_NONE       = 0x00000020, // No source - for testing only
    apOS_INT_32BIT      = 0x7FFFFFFF // Forces compiler to use 32-bit int for enum
}
apOS_INT_oInterruptSource;
```

8.5 Specifying a PrimeCell variant

Some PrimeCell drivers support a range of PrimeCell variants, selected at compile time. Refer to the driver API document to determine if this is the case.

For example, the UART driver supports the PL010, PL011 and the Integrator AP UART.

If installing such a driver, the variant used must be specified in the file `applatfm.h`.

A variant PLxxx for driver *MOD* is chosen by `#define` of a pre-processor constant `apMOD_VERSION`, setting this to value xxx as in Example 18.

Example 18 - Selecting UART type PL011

```
#define apUART_VERSION 11
```

8.6 Enabling timer functionality

By default, timer functionality is not available.

To enable timer functionality, the two functions below must be implemented in `apos/os.c`, and then `apOS_PLATFORM_HAS_TIMERS` in `applatfm.h` must be defined as `TRUE`.

8.6.1 Enabling timers

The function below must be modified to call the timer driver used in order to enable a one-shot timer.

```
PUBLIC apError apOS_TIMER_TimeoutEnable(UWORD32 TimerNumber,  
                                       UWORD32 Microseconds,  
                                       void rCallback(UWORD32),  
                                       UWORD32 CallbackParam)
```

<i>TimerNumber</i>	A specification of which timer is required. A value of zero is passed for a first timer, and increasing numbers if more timers are required at the same time.
<i>Microseconds</i>	The length of time to wait in microseconds
<i>rCallback</i>	A callback function called when the timer elapses
<i>CallbackParam</i>	Parameter to be passed to the user callback routine.

The timer driver must execute the callback after the specified delay, passing it the specified parameter.

8.6.2 Disabling timers

The function below must be modified to call the timer driver used in order to disable a one-shot timer.

```
PUBLIC void apOS_TIMER_TimeoutDisable(UWORD32 TimerNumber)
```

The timer driver must disable the timer specified by the timer number, so preventing the callback function occurring.

There must be no effect if the callback has already occurred.

9 INDIRECTION MODULE

This chapter describes the Application Programmers' Interface (API) to the indirection module [2]. You might want to use or reconfigure this so that you can integrate the software with your own system.

9.1 Internal calls to the Interrupt Controller module

This section describes how the software makes internal calls to the Interrupt Controller module. It contains the following:

9.1.1 The delivered system

The delivered system is preconfigured to use functions named `apOS_INT_*` for internal calls to the Interrupt Controller module. Each of these is functionally equivalent to the corresponding `apINT_*` function in the Interrupt Controller module.

Using these functions provides a hook that can be used to add functionality, such as memory mapping. This makes it easier to enhance the software in future versions.

9.1.2 Interrupt Binding

There are two alternative formats of interrupt binding and associated *Interrupt Service Routines* (ISRs) supported.

Interrupt binding is always performed by `apOS_INT_HandlerSet`, with prototype below:

```
PUBLIC void apOS_INT_HandlerSet(CONST apOS_INT_oInterruptSource oSource,
                               apOS_INT_rServiceRoutine rHandler,
                               UWORD32 Id,
                               apOS_INT_rRawISR rRawHandler);
```

The interrupt controller must associate (bind) the interrupt source with an ISR. This can be either of:

- ◆ `rHandler` specifies a function of prototype below. The third parameter (`Id`) must be stored by the interrupt controller and passed to the function along with the interrupt source when an interrupt occurs.

```
void MOD_FunctionISR(CONST apOS_INT_oInterruptSource oSource, UWORD32 Instance);
```

- ◆ Alternatively, `rRawHandler` specifies a function of prototype below. This function is executed directly from the interrupt vector. Because the source and peripheral instance must be identifiable, the two routines below must be implemented to support the raw ISR.
- `apOS_INT_SourceGet` returns the active interrupt source
- `apOS_INT_InstanceGet` returns the identifier passed (as `Id`) to `apOS_INT_HandlerSet`.

9.1.3 Changing the software

The functions are implemented for the Micropack (INT) or PL190 (VIC) interrupt controllers.

If a different interrupt controller is used, function calls to the interrupt controller will have to be linked to support the functionality described.

9.2 Enabling and disabling interrupts to the processor core

This section describes how the software enables and disables interrupts to the processor core. It contains the following information:

9.2.1 The delivered system

The delivered system uses four functions to enable or disable IRQs or FIQs to the processor core:

- ◆ ap0S_CoreIRQDisable
- ◆ ap0S_CoreIRQEnable
- ◆ ap0S_CoreFIQDisable
- ◆ ap0S_CoreFIQEnable.

Each of these functions is preconfigured to:

1. Read a copy of the CPSR.
3. Change the value of the I or F control bit in the copy, as appropriate.
4. Write the changed copy to the CPSR.

You must be in a privileged mode for this write to succeed.

There is also one function to read the current state of IRQs, returning TRUE if IRQs are enabled (core bit not set).

- ◆ ap0S_CoreIRQGet

These functions are defined in the file `apos\oss.s`.

9.2.2 Changing the software

If you have to call these functions from User mode, you must change their coding appropriately. You must ensure that the processor temporarily enters a privileged mode (such as System mode) so that it can write to the control bits.

9.3 Internal calls to a timer module

This section describes how the software interacts with a timer. The timer might be supplied by ARM or might be a separate driver integrated into the system.

9.3.1 Timeout callbacks

If a module requires a timeout callback, this is implemented through `apOS_TIMER_TimeoutEnable`. This routine specifies a function to be called on elapsing of a specified time. The routine also specifies a timer to use by passing a parameter interpreted as the enumeration of the desired timer (see §8.1).

If the timeout has not elapsed but is no longer required, it can be disabled by a call to `apOS_TIMER_TimeoutDisable` specifying the same timer.

Usage of timeout callbacks is kept to a minimum within the drivers. Where these are required and used, this is documented within the driver API.

«Note» These routines do not check that the timer(s) used have been initialized – it is the responsibility of the system integrator to initialize all required timers before they are used.

This function is defined in the file `apos\apos.h`.

9.3.2 Delay implementation

When a driver has to wait for a timer, it calls the function `apOS_TIMER_Wait`. This executes a blocking wait, returning after the specified time. The implementation is as follows:

- ◆ If interrupts are enabled, `apOS_TIMER_TimeoutEnable` is used, with a polling wait for the callback function to be executed. Timer instance 0 is always used for this function.
- ◆ If interrupts are disabled, or `apOS_TIMER_TimeoutEnable` returns an error (for example if the constant `apOS_PLATFORM_HAS_TIMERS` in `applatfm.h` is defined false), and the C library is available (constant `apOS_CONFIG_USE_NO_CLIB` in `applatfm.h` is not defined false), the C library `clock()` function is used.
The ARM default implementation of `clock()` is semi-hosted and that the implementation might have poor resolution.
- ◆ If interrupts are disabled, or `apOS_TIMER_TimeoutEnable` returns an error, and the C library is not available, the time delay is implemented by a counter.
In this case the delay is unlikely to be accurate.

«Note» This routine does not check that the timer used has been initialized – it is the responsibility of the system integrator to initialize all required timers before they are used.

This function is defined in the file `apos\apos.h`.

9.3.3 Changing the software

You might wish to modify any of these routines to suit your system configuration and the manner in which your system allocates timers. Suggested changes are:

- ◆ Implementing a timer. If the ARM timer or a different driver is being used to support timers, `apOS_TIMER_TimeoutEnable` and `apOS_TIMER_TimeoutDisable` can be modified to request the timeout functions according to the driver API.
- ◆ Modified timer allocation. Drivers usually call `apOS_TIMER_TimeoutEnable` with the enumeration of the desired timer as 0 or 1. `apOS_TIMER_Wait` always passes enumeration 0. The system integrator might choose to assign different timers to the call. For example, if the usage of timer 2 by other system functions is low, this might be used instead.
- ◆ Dynamic timer allocation. If the system can allocate timers according to availability, `apOS_TIMER_TimeoutEnable` might be modified to ignore the enumeration of the desired timer and to allocate a timer at run-time.
- ◆ Improved counter/timer. `apOS_TIMER_Wait` can be implemented by using a counter calibrated specifically for the system used.

10 THE TEST MODULES

This section describes the test application for the software. This builds a simple application using the drivers to perform a system integration test.

10.1 Test Functions

The test module `test.c` implements a number of functions used by the test application and available for other test purposes [3]. They are not called directly by the driver modules.

10.1.1 Constant definitions

The TEST module has the following constant definitions for configuration

- ◆ `apTEST_END_ON_FIRST_FAILURE` configures whether all testing ends on the first failure of a test
 - if TRUE, all testing ends if any module test returns a failure result
 - if FALSE, testing continues even if a module test returns a failure result.
- ◆ `apTEST_VERBOSE` configures whether to print verbose output from the tests
 - if TRUE, all diagnostic messages are printed
 - if FALSE, a diagnostic message is printed only if it terminates testing of a module.
- ◆ `apTEST_INTERACTIVE` configures whether testing is interactive.
 - if TRUE, the user can press a key to continue a test after a pause and can assess the success or failure of a test.
 - if FALSE, testing is not interactive, and the software can pause but not request input.

The defaults for the system are:

- ◆ `apTEST_END_ON_FIRST_FAILURE = FALSE`
- ◆ `apTEST_VERBOSE = TRUE` unless `apOS_CONFIG_USE_NO_CLIB` is defined as TRUE
- ◆ `apTEST_INTERACTIVE = TRUE` unless `apOS_CONFIG_USE_NO_CLIB` is defined as TRUE

10.1.2 Testing with a timeout handler

The TEST module supports a macro `apTEST_TIMEOUT` that allows a test to be performed with a timeout handler.

When you call this function:

1. It sets the global variable `apTEST_TestInProgress` to `TRUE`.
2. It sets up the specified timer, printing the timeout as a diagnostic message, and sets a callback handler to regain control if there is a timeout.
3. It calls the `testFunc`, passing the peripheral identifier if one was specified.

The `testFunc` must then set up registers as necessary, and return so that the test is controlled by interrupts. When the test is complete, the interrupt service routine that is running the tests must set `apTEST_TestInProgress` to `FALSE`.

If you must keep program flow inside `__testFunc`, then it can contain a polling loop as below:

```
while(apTEST_TestInProgress)
{ }
```

4. On completion of the test, or on a timeout, it disables the timer.
5. If the test times out, it:
 - a. Prints the timeout message.
 - b. Returns the timeout result, which is typically `apTEST_FAIL`.

10.1.3 Changing the software

You might want to change the test software routines. For example:

- ◆ You might configure the test software to end testing on the first failure of a test. See `apTEST_END_ON_FIRST_FAILURE` in §10.1.1
- ◆ You might want to remove the dependence on a keyboard for input. You can either stop the tests from running interactively (see `apTEST_INTERACTIVE` in §10.1.1), or you can change the input device by changing the source code. For example, you might choose to use a switch on the development platform.
- ◆ You might want to change how the tests output their results. For example, you might change `apTEST_Print` to illuminate a red LED when a test fails, and illuminate a green LED when a test passes.

10.2 Bootstrap code

10.2.1 Stack and Heap

Stack and heap sizes are specified in `aptest/sysdefs.h`, and the default settings are as below.

Mode	FIQ	IRQ	ABT	UND	SVC	USR
Stack size	1k	4k	1k	1k	4k	16k

Heap Size: 16k

The rules for placement of the stack and heap are:

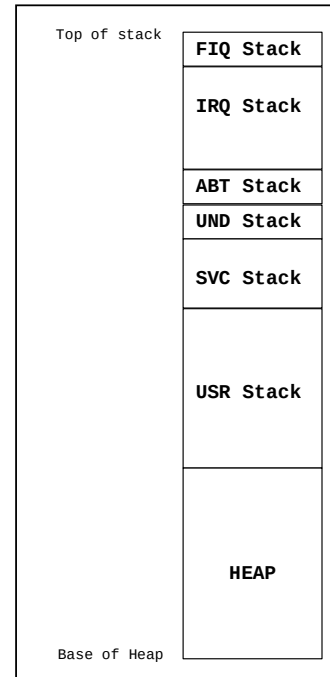
Non-Scatterloaded code

- ◆ Heap Base placed directly after the code and data, at the linker-generated label `Image$$ZI$$Limit`.
- ◆ Stacks placed above heap, based on heap size defined above

Scatterloaded code

The location of stack and heap depend on the execution regions declared in the scatterloading file.

- ◆ if a region named *HEAP* exists, the heap is placed in this region
- ◆ otherwise, if a region named *RAM* exists, the heap is placed in this region
- ◆ if a region named *STACK_END* exists, the stacks are placed *below* this region, so the top stack location is the byte *before* this region starts
- ◆ otherwise stacks are placed above heap, based on heap size defined above.



10.2.2 Changing the stack and heap settings

The values in `aptest/sysdefs.h` can be edited as desired to reflect new stack and heap requirements.

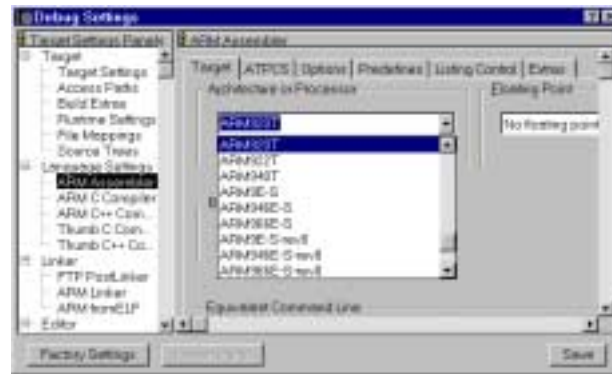
10.2.3 Cache/MMU/MPU support

The test application sets up the caches and *Memory Management/Protection Unit* (MMU/MPU) () as appropriate for the core being used. This is set in the bootstrap code according to the pre-defined assembler constant {CPU}.

10.2.4 Changing the cached core

In CodeWarrior under ADS, select the desired core for assembly as in Example 19. This sets the {CPU} constant automatically.

Example 19 – Selecting a core type in ADS



If using a different toolchain, the {CPU} constant can be defined on the command line, or the CACHE_BUILD constant set in boot.s, to the name of the core to select the cache enabling code appropriate to that core.

Example 20 – Setting cache configuration for ARM720T

```
CACHE_BUILD SETS "ARM720T"
```

The supported cores are:

- ◆ ARM720T
- ◆ ARM740T
- ◆ ARM920T
- ◆ ARM940T.

10.2.5 Setting up memory management

Where an MMU or MPU are present in the selected core, a configuration block in file `apt est /mmu_mpu . s` is required to define the memory map and attributes.

This data is decoded by the bootstrap code to set up the MMU or MPU according to the core selected. This allows the code to be switched easily between cores.

The data in this file consists of a sequence of records as below:

Width	Datum
Word	Start address of memory region (in MB)
Word	Width of memory region (in MB)
Byte	Is memory region cacheable (non-zero if cacheable)
Byte	Is memory region bufferable (non-zero if bufferable)
Byte	Is memory region readable (non-zero if read access allowed)
Byte	Is memory region writeable (non-zero if write access allowed)

Where a memory location lies in more than one specified region, the last listed has priority.

10.2.6 Changing the memory configuration

To change the memory configuration, edit the file `aptest/mmu_mpu.s`.

Memory regions can be appended, replaced or deleted as long as each adheres to the correct format.

Example 21 shows an example memory description. The associated memory map is:

- ◆ Flash ROM. A 128 MB block at `0x20000000`. This is cached and read-only.
- ◆ RAM. A 256 MB block at `0x00000000`. This is cached and read-write.
- ◆ The remaining areas in the memory map might contain I/O devices. They are uncached and read-write.

«Note» The label `MMU_MPU_RegionData` must be present at the start of the data block and the label `MMU_MPU_RegionDataEnd` must be at the end.

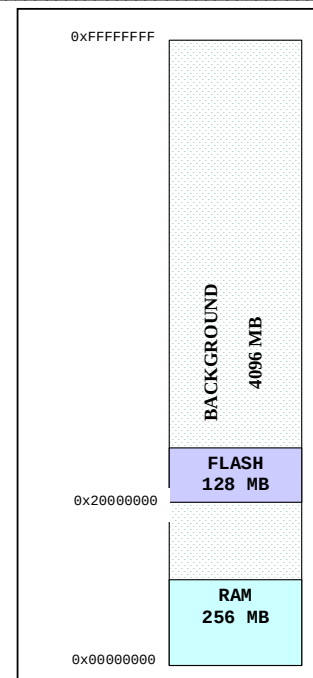
Example 21 – Example memory map

```
MMU_MPU_RegionData
; //Background region covers all of memory
DCD 0 ; //Starting section in MB
DCD 4096 ; //Width in MB
DCB 0,0,1,1 ; //Cache, buffer, read, write

; //RAM region covers normal memory
DCD 0 ; //Starting section in MB
DCD 256 ; //Width in MB
DCB 1,0,1,1 ; //Cache, buffer, read, write

; //ROM region covers flash
DCD 0x200 ; //Starting section in MB
DCD 128 ; //Width in MB
DCB 1,0,1,0 ; //Cache, buffer, read, write

MMU_MPU_RegionDataEnd
```



10.2.7 Interrupt Controllers

The test application initializes one or more interrupt controllers according to the settings of the pre-processor constant `apINT_VERSION` defined in `apos/apintcfg.h`.

10.2.7.1 Micropack controller

If `apINT_VERSION` is set to `apVERSION_INTEGRATOR`, `apVERSION_ARMULATOR` or `apVERSION_VIC_ON_LM`, a basic INT (Micropack) controller is initialized with instance `apOS_INT_0` and base address `apOS_SYSTEM_BASE_INT`.

10.2.7.2 Logic Module

If developing on the Integrator platform, as indicated by the `apOS_PLATFORM_INTEGRATOR` constant being `TRUE`, the code does the following:

- ◆ Check that a logic module is connected.
- ◆ If `apINT_VERSION` is set to `apVERSION_INTEGRATOR`, the logic module is assumed to contain a basic INT (Micropack) controller.
 - this is initialized with instance `apOS_INT_LM` and base address `apOS_SYSTEM_BASE_INTLM`.
 - the interrupt controller is chained onto the dispatcher of the primary controller as interrupt source `apOS_INT_EXP0`.
- ◆ If `apINT_VERSION` is set to `apVERSION_VIC_ON_LM`, the logic module is assumed to contain a Vectored (VIC) controller. Interrupt source `apOS_INT_EXP0` is enabled on the main FIQ interrupt controller so that the VIC controller generates FIQs.

10.2.7.3 Vectored controller

If `apINT_VERSION` is set to `apVERSION_VECTORED` or `apVERSION_VIC_ON_LM`, a Vectored (VIC) controller is initialized with instance `apOS_VIC_0` and base address `apOS_SYSTEM_BASE_VIC`.

10.2.8 Timers

If pre-processor constant `apOS_PLATFORM_HAS_TIMERS` is `TRUE`, a single timer with instance `apOS_TIMER_0` and base address `apOS_SYSTEM_BASE_TIMER0` is initialized for use in timeouts.

This requires a `TIMER` module that is compatible with the PrimeCell driver initialization API.

10.3 Exception vectors

The exception vectors are set in file `aptest/vectors.s`.

«Note» If the code is loaded to address 0x0, or is scatterloaded, care must be taken to ensure that the vectors are at address 0x0 in the execution view.

If the code is loaded to any other address, for use with a debugger, the bootstrap copies the vectors to address 0x0.

Three vectors are implemented:

10.3.1 Reset vector

The reset vector calls the bootstrap code in `aptest/boot.s`.

10.3.2 IRQ vector

The IRQ vector performs one of the following, depending on the setting of pre-processor constants in `apos/apintcfg.h`.

- ◆ If `ap0S_CONFIG_VIC_AT_0xFFFFF000` is non-zero, there is a vectored controller present at base address `0xFFFFF000`. For speed, the IRQ vector loads directly from the vectored address register. See the PL190 documentation for details.
- ◆ If `apINT_VERSION` is set to `apVERSION_VECTORED`, there is a Vectored controller only, and the IRQ vector jumps to the IRQ dispatcher for the vectored controller.
- ◆ Otherwise, the IRQ vector jumps to the dispatcher for the INT (Micropack) controller.

10.3.3 FIQ vector

The FIQ vector performs one of the following, depending on the setting of pre-processor constants in `apos/apintcfg.h`.

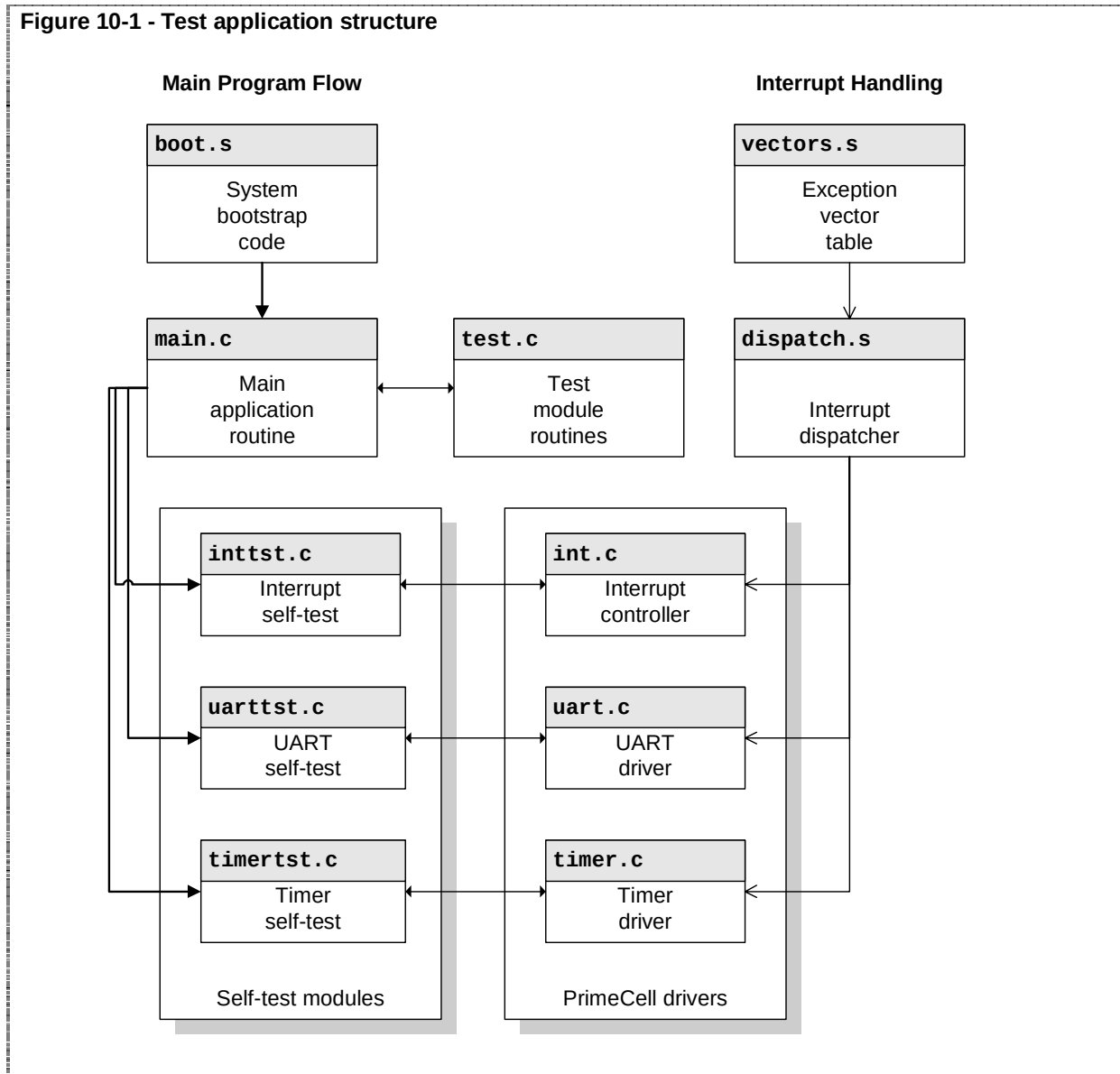
- ◆ If `ap0S_CONFIG_VIC_AT_0xFFFFF000` is non-zero, there is a vectored controller present at base address `0xFFFFF000`. For speed, the FIQ vector loads directly from the vectored address register. See the PL190 documentation for details.
- ◆ If `apINT_VERSION` is set to `apVERSION_VIC_ON_LM`, there is a Vectored controller on the logic module, and the FIQ vector jumps to the IRQ dispatcher for the vectored controller.
- ◆ If `apINT_VERSION` is set to `apVERSION_VECTORED`, there is a Vectored controller only, and the FIQ vector jumps to the FIQ dispatcher for the vectored controller.
- ◆ Otherwise, the FIQ vector jumps to the FIQ dispatcher for the INT (Micropack) controller.

10.4 Test Application

10.4.1 The delivered system

The delivered system is configured to build a standalone application using the test modules supplied with each driver. The application structure is as shown in Figure 10-1.

Figure 10-1 - Test application structure



10.4.2 Self-test modules

Each driver is supplied with a self-test module, which is a single file named *MODtst.c*, where *MOD* is the name of the driver. For example, the UART driver has a self-test module named *uarttst.c*.

The purpose of the self-test module is to use the driver to perform a basic test on the PrimeCell. The self-test is performed without the requirement for external equipment, beyond a basic interface where appropriate.

The level of testing is thus fairly restricted and is documented for each driver [7].

The self-test modules use the test functions (see §10.1) to output informational messages and return a PASS or FAIL result. The main routine in each case has the prototype in Example 22.

Example 22 - Self-test function

```
PROTECTED apTEST_eResult apUART_eTest
(
    UWORD32 RegBase,
    CONST apOS_INT_oInterruptSource * CONST pInt
);
```

10.4.3 Changing the software

You might want to change the test application behavior. For example:

- ♦ You might configure the test software to end testing on the first failure of a test. See *apTEST_END_ON_FIRST_FAILURE* in §10.1.1
- ♦ You might want to remove the dependence on a keyboard for input. To run all the tests without user intervention, see *apTEST_INTERACTIVE* in §10.1.1).

10.5 Specifying the modules to test

10.5.1 TEST_ALL_MODULES

The set of PrimeCells to be tested is specified by a macro *TEST_ALL_MODULES* defined in *applatfm.h*. This is defined as a list of peripherals as in Example 23.

Example 23 - Determining modules to test

```
#define TEST_ALL_MODULES \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,0), apOS_SYSTEM_BASE_TIMER0, apOS_INT_TIMER0); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,1), apOS_SYSTEM_BASE_TIMER1, apOS_INT_TIMER1); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,2), apOS_SYSTEM_BASE_TIMER2, apOS_INT_TIMER2); \
    TEST_MODULE(KMI, 0, apOS_INSTANCE(KMI,0), apOS_SYSTEM_BASE_KMI0, apOS_INT_KMI0); \
    TEST_MODULE(KMI, 0, apOS_INSTANCE(KMI,1), apOS_SYSTEM_BASE_KMI1, apOS_INT_KMI1); \
    TEST_MODULE(UART, 0, apOS_INSTANCE(UART,0), apOS_SYSTEM_BASE_UART0, apOS_INT_UART0); \
    TEST_MODULE(UART, 0, apOS_INSTANCE(UART,1), apOS_SYSTEM_BASE_UART1, apOS_INT_UART1); \
    TEST_MODULE(RTC, 30, apOS_INSTANCE(RTC,0), apOS_SYSTEM_BASE_RTC, apOS_INT_RTC);
```

Each line of the TEST_ALL_MODULES macro except the last must have a terminating backslash (escape) character, to ensure that they form part of the same macro. Care must be taken in commenting out sections of the macro. The pre-processor removes commented sections, thereby possibly losing the backslash characters. Example 24, in which the user has attempted to remove the KMI tests by adding C++ style comment markers, does not compile correctly.

Example 24 – Incorrect manipulation of the TEST_ALL_MODULES macro

```
#define TEST_ALL_MODULES \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,0), apOS_SYSTEM_BASE_TIMER0, apOS_INT_TIMER0); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,1), apOS_SYSTEM_BASE_TIMER1, apOS_INT_TIMER1); \
    TEST_MODULE(TIMER, 0, apOS_INSTANCE(TIMER,2), apOS_SYSTEM_BASE_TIMER2, apOS_INT_TIMER2); \
    // TEST_MODULE(KMI, 0, apOS_INSTANCE(KMI,0), apOS_SYSTEM_BASE_KMI0, apOS_INT_KMI0); \
    // TEST_MODULE(KMI, 0, apOS_INSTANCE(KMI,1), apOS_SYSTEM_BASE_KMI1, apOS_INT_KMI1); \
    TEST_MODULE(UART, 0, apOS_INSTANCE(UART,0), apOS_SYSTEM_BASE_UART0, apOS_INT_UART0); \
    TEST_MODULE(UART, 0, apOS_INSTANCE(UART,1), apOS_SYSTEM_BASE_UART1, apOS_INT_UART1); \
    TEST_MODULE(RTC, 30, apOS_INSTANCE(RTC,0), apOS_SYSTEM_BASE_RTC, apOS_INT_RTC);
```

10.5.2 TEST_MODULE

The TEST_MODULE macro takes five parameters as follows:

- ◆ Name of driver to test (e.g. UART)
- ◆ The PrimeCell number of the driver (e.g. 11 for the UART PL011). Many PrimeCells support identification registers and this number is checked against these registers. If the PrimeCell does not support this, the value 0 must be specified.
- ◆ The enumeration of the PrimeCell to test (see §8.1), specified using the apOS_INSTANCE() macro
- ◆ The base address of the PrimeCell being tested as an enumeration (see §8.2)
- ◆ The interrupt source assigned to the PrimeCell being tested as an enumeration (see §8.4, §10.5.4, §10.5.5)

10.5.3 apOS_INSTANCE

The apOS_INSTANCE macro is used to specify the PrimeCell to test (see §8.1). It takes two parameters as follows:

- ◆ name of driver to test (e.g. UART)
- ◆ number of that PrimeCell to test.

In the normal build, this macro combines the two arguments *MOD* and *N* to produce an enumerator apOS_MOD_N.

If the constant apOS_NO_STATIC_STATE is declared (see §7.5.2), an area of memory is assigned on the heap for the test using malloc.

10.5.4 TEST_NOINT

If the PrimeCell has no assigned interrupts, the TEST_NOINT constant must be passed as the last parameter to TEST_MODULE.

10.5.5 TEST_LIST

If the PrimeCell has multiple assigned interrupts, the TEST_LIST macro is used to specify the last parameter to TEST_MODULE.

TEST_LIST takes two parameters, each of which can be an interrupt source assigned to the PrimeCell being tested as an enumeration (see §8.4) or another TEST_LIST macro. Example 25 shows how a test could be specified for a hypothetical PrimeCell PL123 of type MOD, with three interrupt sources.

Example 25 - Specifying three interrupt sources

```
#define TEST_ALL_MODULES \  
    TEST_MODULE(MOD, 123, apOS_MOD_0, apOS_SYSTEM_BASE_MOD, \  
                TEST_LIST(apOS_INT_MOD1, \  
                           TEST_LIST(apOS_INT_MOD2, apOS_INT_MOD3) \  
                           ) \  
    );
```

11 COMMON MODULE

11.1 Purpose of the Common module

The Common module does not correspond to any peripheral. It instead provides type definitions and macros that are used across the project and that implement the coding standard used. Having a central repository for these type definitions and macros makes it simpler to `#include` them into other modules.

The full descriptions of these items in the Common module API [\[4\]](#) .

11.2 Macros defining scope

11.2.1 PUBLIC

Sets the scope of the following item as the public API.

11.2.2 PRIVATE

Sets the scope of the following item to be limited to the module.

11.2.3 PROTECTED

Sets the scope of the following item as public, but the item is not included in the API.

11.3 Further keywords

11.3.1 PACKED

Defines a packed structure.

11.3.2 INLINE

Specifies a routine to be in line.

11.3.3 CONST

Defines a constant item.

11.3.4 IRQ

Defines a function as being an Interrupt Service Routine called directly from the IRQ vector.

11.4 Additional constants

11.4.1 TRUE

Standard Boolean value.

11.4.2 FALSE

Standard Boolean value.

11.4.3 aNULL

A null pointer to anything except for a routine.

11.4.4 aRNULL

A null pointer to a routine.

11.5 Additional macros

11.5.1 IGNORE

Indicates that the following variable is not used (for expansion).

11.6 Primitive types

11.6.1 BOOL

A Boolean value, to have one of the constant values TRUE or FALSE (see §11.3.4).

For speed on the ARM architecture, this is implemented as a 32 bit value.

11.6.2 BYTE8

A signed byte.

11.6.3 UBYTE8

An unsigned byte.

11.6.4 HWD16

A signed 16-bit halfword.

11.6.5 UHWD16

An unsigned 16-bit halfword.

11.6.6 WORD32

A signed 32-bit word.

11.6.7 UWORD32

An unsigned 32-bit word.

11.7 General function types

11.7.1 apTYPE_rCallback

Function type for general-purpose callback, typically used for handling errors or interrupts.

11.8 Debug message macros

The Common module has the following constants that are used to control debug messages:

- ◆ apDEBUG
- ◆ apDEBUG_LEVEL
- ◆ apDEBUG_BUFFER
- ◆ apDEBUG_BUFFER_SIZE
- ◆ apDEBUG_PRINTF_MAX
- ◆ apDEBUG_ENABLED.

and the following functions that are used to output debug messages:

- ◆ apDEBUG_INFO
- ◆ apDEBUG_WARN
- ◆ apDEBUG_CRIT.

11.8.1 Constants for enabling debug

11.8.1.1 apDEBUG

Constant for enabling output of debug messages:

- ◆ If apDEBUG is defined, debug messages are output. The apDEBUG_LEVEL constant sets the level of debug messages that are output.
- ◆ If apDEBUG is not defined, debug messages are not output. The following macros become null:
 - apASSERT
 - apDEBUG_INFO
 - apDEBUG_WARN
 - apDEBUG_CRIT.

11.8.1.2 apDEBUG_ENABLED

Constant used internally to store whether debug message output is enabled:

- ◆ if apDEBUG_ENABLED is defined, debug messages are output.
- ◆ if apDEBUG_ENABLED is not defined, debug messages are not output.

11.8.1.3 apDEBUG_LEVEL

Constant for setting the level of debug messages that are output:

- ◆ if apDEBUG_LEVEL is defined, its value sets the level of debug messages that are output
- ◆ if apDEBUG_LEVEL is not defined, the system outputs all levels of debug messages.

apDEBUG_LEVEL	apDEBUG_INFO	apDEBUG_WARN	apDEBUG_CRIT
Undefined	Yes	Yes	Yes
0	Yes	Yes	Yes
1	No	Yes	Yes
2	No	No	Yes

11.8.2 Constants for enabling buffered debug

11.8.2.1 apDEBUG_BUFFER

- ◆ If apDEBUG_BUFFER is not defined, debug messages are sent to `stderr`, which is implemented through the ARM C Library as semi-hosting calls, or can be retargeted as desired.
- ◆ If apDEBUG_BUFFER is defined, debug messages are sent to a buffer accessed through a global pointer named apDEBUG_Buffer. You must then define the following variables in your own code:

```
PUBLIC int  apDEBUG_Counter=0;
PUBLIC char apDEBUG_Working[apDEBUG_PRINTF_MAX];
PUBLIC char apDEBUG_Buffer[apDEBUG_BUFFER_SIZE]="<#>";
```

Debug messages are written into consecutive locations in the buffer, returning to the start if there is insufficient remaining free space. No debug message is ever split, but a message that is longer than the buffer is truncated by the end of the buffer. The end of the latest message is marked by the token `<#>`.

◆

11.8.2.2 apDEBUG_BUFFER_SIZE

Constant for defining the size of the debug message buffer. This is ignored if apDEBUG_BUFFER is not defined.

- ◆ if apDEBUG_BUFFER_SIZE is defined, its value sets the length of the debug message buffer apDEBUG_Buffer, in bytes.
- ◆ if apDEBUG_BUFFER_SIZE is not defined, the length of the debug message buffer apDEBUG_Buffer is set to 1024 bytes.

11.8.2.3 apDEBUG_PRINTF_MAX

Constant for defining the maximum length of debug message written to the buffer. This is ignored if apDEBUG_BUFFER is not defined.

- ◆ if apDEBUG_PRINTF_MAX is defined, its value sets the maximum length of debug message written to the debug message buffer apDEBUG_Buffer, in bytes.
- ◆ if apDEBUG_PRINTF_MAX is not defined, the maximum length of debug message written to the debug message buffer apDEBUG_Buffer is set to 128 bytes.

If any debug messages exceed this maximum length, you get unpredictable results.

11.8.3 Macros for debug output

11.8.3.1 apDEBUG_INFO

Outputs a debug information message

Syntax

```
void apDEBUG_INFO(format, argument)
```

format The format for printing, passed as the first argument to `printf`.

argument The argument for printing, passed as the second argument to `printf`.

Usage

This function outputs a debug information message. The two arguments are passed to the function `printf`. Typical usage might be:

```
apDEBUG_INFO("Current status: %x\n", StatusWord);
```

This function is implemented as a macro.

11.8.3.2 apDEBUG_WARN

Outputs a debug warning message

Syntax

```
void apDEBUG_WARN(format, argument)
```

format The format for printing, passed as the first argument to `printf`.

argument The argument for printing, passed as the second argument to `printf`.

Usage

This function outputs a debug information message. The two arguments are passed to the function `printf`. Typical usage might be:

```
apDEBUG_WARN("Current status: %x\n", StatusWord);
```

11.8.3.3 apDEBUG_CRIT

Outputs a critical debug message

Syntax

```
void apDEBUG_CRIT(format, argument)
```

format The format for printing, passed as the first argument to `printf`.

argument The argument for printing, passed as the second argument to `printf`.

Usage

This function outputs a debug information message. The two arguments are passed to the function `printf`. Typical usage might be:

```
apDEBUG_CRIT("Current status: %x\n", StatusWord);
```

This function is implemented as a macro.

11.9 Bitfield operation functions implemented as macros

The Common module has the following bitfield operation functions implemented as macros. These are in `apbitops.h`.

- ◆ `apBITS_ALL`
- ◆ `apBYTE_STREAM_ENDIAN_ADJUST`
- ◆ `apHWD_STREAM_ENDIAN_ADJUST`
- ◆ `apBIT_MASK`
- ◆ `apBIT_CLEAR`
- ◆ `apBIT_GET_UNSHIFTED`
- ◆ `apBIT_GET`
- ◆ `apBIT_SET`
- ◆ `apBIT_SET_NOREAD`
- ◆ `apBIT_BUILD`
- ◆ `apBIT_MASK_FIELD`
- ◆ `apBIT_CLEAR_FIELD`
- ◆ `apBIT_GET_FIELD_UNSHIFTED`
- ◆ `apBIT_GET_FIELD`
- ◆ `apBIT_SET_FIELD`
- ◆ `apBYTE_SET_FIELD`
- ◆ `apBIT_SET_FIELD_NOREAD`
- ◆ `apBIT_BUILD_FIELD`.

These macros have public scope so that other modules can use them. Although they are a part of the Common module API, they are typically only used within drivers. The most common reason for using these macros is if you add a driver.

11.9.1 General Items

11.9.1.1 apBITS_ALL

Generic constant for accessing all bits of a 32-bit word.

11.9.1.2 apBYTE_STREAM_ENDIAN_ADJUST

- ◆ If the `__BIG_ENDIAN` constant is defined, this reverses the order of bytes within the word variable passed as an argument.
- ◆ Otherwise, the variable is unchanged

The macro is present to correct endian issues in multi-byte FIFOs.

11.9.1.3 apHWD_STREAM_ENDIAN_ADJUST

- ◆ If the `__BIG_ENDIAN` constant is defined, this reverses the order of half-words within the word variable passed as an argument.
- ◆ Otherwise, the variable is unchanged

The macro is present to correct endian issues in multi-half-word FIFOs.

11.9.2 Single bitfield macros

These macros refer to bitfields by a single parameter. This parameter is not used directly. Instead, it is expanded into two constants, that must be previously defined, as in Example 26, which returns a bit mask for bits 4 to 11.

bwParam The width of the bitfield, in bits.

bsParam The left shift of the bitfield, in bits.

Example 26 – Single bitfield macro expansion

```
#define bsDATA_BITS 4          /*define the lowest bit*/
#define bwDATA_BITS 8          /*define the width in bits*/
MyMask=apBIT_MASK(DATA_BITS); /*returns 0x00000FF0 - mask 8 bits wide from bit 4*/
```

11.9.2.1 apBIT_MASK

Returns the bitmask for a predefined bitfield.

11.9.2.2 apBIT_CLEAR

Clears a predefined bitfield in the specified data.

11.9.2.3 apBIT_GET_UNSHIFTED

Gets a predefined bitfield from specified data, unshifted.

11.9.2.4 apBIT_GET

Gets a predefined bitfield from specified data, shifted into the lowest bits.

11.9.2.5 apBIT_SET

Sets a predefined bitfield in specified data to a specified value.

11.9.2.6 apBIT_SET_NOREAD

Sets a predefined bitfield in specified data to a specified value, without first reading the data. This is useful for registers that are read-sensitive, such as interrupt registers. This function clears all other bits in the data.

11.9.2.7 apBIT_BUILD

Builds a predefined bitfield, setting it to a specified value.

11.9.3 Dual bitfield macros

These macros refer to bitfields by two parameters. These separately specify the bit shift and width as in Example 27, which returns a bit mask for bits 4 to 11.

WidthParam The width of the bitfield, in bits.

ShiftParam The left shift of the bitfield, in bits.

Example 27 – Dual bitfield macro expansion

```
#define DATA_BITS_SHIFT 4      /*define the lowest bit*/  
#define DATA_BITS_WIDTH 8     /*define the width in bits*/  
MyMask=apBIT_MASK_FIELD(DATA_BITS_WIDTH,DATA_BITS_SHIFT);  
/*returns 0x00000FF0 - mask 8 bits wide from bit 4*/
```

11.9.3.1 apBIT_MASK_FIELD

Returns the bitmask for a defined bitfield.

11.9.3.2 apBIT_CLEAR_FIELD

Clears a defined bitfield in the specified data.

11.9.3.3 apBIT_GET_FIELD_UNSHIFTED

Gets a defined bitfield from specified data, unshifted. It returns the bitfield unshifted, with all other bits clear. If you want the returned bitfield to be shifted into the lowest bits, you must instead call apBIT_GET_FIELD.

11.9.3.4 apBIT_GET_FIELD

Gets a defined bitfield from specified data, shifted into the lowest bits, with all other bits clear. If you want the returned bitfield to be unshifted, you must instead call apBIT_GET_FIELD_UNSHIFTED.

11.9.3.5 apBIT_SET_FIELD

Sets a defined bitfield in specified data to a specified value.

11.9.3.6 apBYTE_SET_FIELD

Sets a defined bitfield in specified data to a specified value. It preserves all other bits. Unlike the other bit operations, this function is for use with data of type BYTE8.

11.9.3.7 apBIT_SET_FIELD_NOREAD

Sets a defined bitfield in specified data to a specified value, without first reading the data. This is useful for registers that are read-sensitive, such as interrupt registers. This function clears all other bits in the data.

11.9.3.8 apBIT_BUILD_FIELD

Builds a defined bitfield, setting it to a specified value.

11.10 Other inline functions implemented as macros

The Common module has the following other inline functions implemented as macros:

- ◆ MAX
- ◆ MIN
- ◆ apASSERT.

11.10.1 MAX

Returns the maximum of the two values it is passed.

11.10.2 MIN

Returns the minimum of the two values it is passed.

11.10.3 apASSERT

Calls the assert macro if the software is built to use asserts.

Usage

This function calls the assert macro if any of the following are true:

- ◆ apOS_CONFIG_USE_ASSERTS is defined and is equal to 1
- ◆ apDEBUG is defined

11.11 Header Files for C and assembler

Some header files have definitions that must be included both in C and in assembler. To enable this, they use a special format. Macros then convert the special header files to valid C or assembler source, as appropriate.

This section describes the format used.

11.11.1 Translation header files

If a header file is coded in this format, it must be *translated* by previously including either the `cmacros.h` file, if the header file is to be included in a C file, or the `asmacros.h` file if included in an assembler file. The functionality of these files is not documented here.

A example of the use of such a header file in a C file (`apint/int.c`) is shown in Example 28 and in an assembler file (`apint/dispatch.s`) is shown in Example 29.

Example 28 - Inclusion of common format header in C file

```
#include "../apcommon/cmacros.h"    /* This is required to include apintcfg.h */
#include "apintcfg.h"                /*Interrupt configuration file*/
```

Example 29 - Inclusion of common format header in assembler file

```
; /* Include the macros to allow common headers */
INCLUDE ../apcommon/asmacros.h
; /* Include the peripheral versions - for the interrupt controller type. */
INCLUDE ../apint/apintcfg.h
```

11.11.2 Header files affected

Only one header file is in this format. This is the `apintcfg.h` file, which primarily holds configuration data for the interrupt controller. This data is used by:

- ◆ the interrupt controller C source file `int.c`
- ◆ the interrupt dispatcher assembler source file `dispatch.s`
- ◆ the system integration assembler source file `oss.s`

11.11.3 Preprocessor instructions

All preprocessor instructions, such as a `#define`, have a special format. The `#` that introduces the control line:

- ◆ must be in the first column
- ◆ must be followed by a space.

See Example 30.

Example 30 - Compiler control lines in the `apintcfg.h` file

```
# ifndef APINTCFG_H
# define APINTCFG_H
```

11.11.4 Comments

Comments have a special format:

- ◆ The comments must be marked as usual for a C program.
- ◆ Each line of comments must be preceded by the string “`COMMENT`”.

For examples of some valid formats, see Example 31.

Example 31 - Comments in the `apintcfg.h` file

```
COMMENT // $Copyright:
COMMENT // -----
COMMENT // This confidential and proprietary software may be used only as
COMMENT // authorised by a licensing agreement from ARM Limited
COMMENT // (C) COPYRIGHT 2000 ARM Limited
COMMENT // ALL RIGHTS RESERVED
```

12 INTERRUPT CONTROLLER

This chapter outlines the Interrupt Controller module. The API is detailed in a separate document [\[5\]](#).

12.1 Purpose of the Interrupt Controller module

The Interrupt Controller module is released in object form with each PrimeCell driver.

The intention of this release is to allow the integration test software (see §10.2.7) to be developed and run.

The functions of the object release of the Interrupt Controller are:

- ◆ to initialize the Interrupt Controller at system startup
- ◆ to provide facilities so that other modules can enable, disable, and clear interrupts
- ◆ to provide functions to control the Interrupt Controller.
- ◆ to add pre-dispatch and post-dispatch handlers, for tasks such as preserving process state around interrupts

12.2 Initializing the Interrupt Controller

The interrupt controller is initialized in the same manner as other drivers, by a call to `apINT_Initialise`.

This does not enable the core interrupts, which must be enabled after the controller is initialized as in Example 32

Example 32 - Initializing an interrupt controller

```
apINT_sInitialData sInitial;
// Initialize the interrupt controller, passing initialization data
sInitial.eSense    = apINT_DEFAULT_SENSE;
sInitial.Priority  = 16;
apINT_Initialize(apOS_INT_0, (apOS_System_eBaseAddress)RegBase, 0, 0, &sInitial);
// Perform other initialization with interrupts disabled, as desired
.
.
// Enable both IRQ and FIQ interrupts
apOS_CoreIRQEnable();
apOS_CoreFIQEnable();
```

Where multiple chained controllers are used, they must be initialized and then chained together using `apINT_HandlerChain`, specifying the interrupt source that is used by the slave controller, as in Example 33.

Note that the encoding of an interrupt source (here `apOS_INT_EXPANSION`) must specify both the controller and the source for that controller.

A controller can be removed from a chain by using `apINT_HandlerClear`.

Example 33 - Chained interrupt controllers

```
apINT_sInitialData sInitial;
// Initialize the master interrupt controller, passing initialization data
    sInitial.eSense    = apINT_DEFAULT_SENSE;
    sInitial.Priority = 16;
    apINT_Initialize(apOS_INT_MASTER, apOS_SYSTEM_BASE_INT_MASTER, 0, 0, &sInitial);
// Initialize the slave interrupt controller, passing initialization data
    apINT_Initialize(apOS_INT_SLAVE, apOS_SYSTEM_BASE_INT_SLAVE, 0, 0, &sInitial);
// Chain the slave to the relevant interrupt source
    apINT_HandlerChain(apOS_INT_EXPANSION, apOS_INT_SLAVE);
// Enable the interrupt source
    apINT_InterruptEnable(apOS_INT_EXPANSION);
// Perform other initialization with interrupts disabled, as desired
    .
    .
// Enable both IRQ and FIQ interrupts
    apOS_CoreIRQEnable();
    apOS_CoreFIQEnable();
    .
    .
// At some later point, remove the slave from the chain
    apINT_HandlerClear(apOS_INT_EXPANSION);
```

12.3 Binding an interrupt source to a handler

Binding (or associating) an interrupt source to a handler is performed with the following stages:

- ◆ Use `apINT_HandlerSet` to specify the routine to be executed by the interrupt dispatcher. It is not necessary explicitly to disable the interrupt before setting the handler.
- ◆ Use `apINT_HandlerEnable` to make the interrupt source active.

Unbinding (or removing) an interrupt source from a handler is performed with the following stages:

- ◆ use `apINT_HandlerDisable` to make the interrupt source active
- ◆ use `apINT_HandlerSet` with an argument of `aNULL` as a handler to associate a null handler with the source.

An example of binding and unbinding a source is shown in Example 34. For a further example, see the source file for Interrupt Controller testing (`apint\inttst.c`).

Example 34 - Binding and unbinding sources

```
//Prototype for the Interrupt Service Routine
extern void MyServiceRoutine(CONST apOS_INT_oInterruptSource oSource, UWORD32
Parameter);
.
.
//Associate the ISR with the UART interrupt source and identify it as UART #0
apINT_HandlerSet(apOS_INT_UART0, & MyServiceRoutine, apOS_UART_0);
apINT_HandlerEnable(apOS_INT_UART0);
.
.
//Unbind this source - the last parameter is not used
apINT_HandlerDisable(apOS_INT_UART0);
apINT_HandlerSet(apOS_INT_UART0, aNULL, 0);
```

12.4 Using the interrupt dispatcher

The interrupt dispatcher module **dispatch.s** has two entry points:

- ◆ apINT_IRQDispatcher
- ◆ apINT_FIQDispatcher

These must be linked to the IRQ and FIQ exception vectors in a suitable fashion – one possible means is shown in Example 35

Example 35 - Linking exception vectors

```
;//Place this code at address 0x00000000
AREA    |VECTOR|, CODE, READONLY
CODE32
ENTRY
IMPORT  apINT_IRQDispatcher
IMPORT  apINT_IRQDispatcher

VECTOR_StartAddress          ;// Start of ARM processor vectors
    LDR    pc, VECTOR_Reset    ;// 00 - Reset
    LDR    pc, VECTOR_Undef    ;// 04 - Undefined instructions
    LDR    pc, VECTOR_SWI      ;// 08 - SWI instructions
    LDR    pc, VECTOR_PAbort   ;// 0C - Instruction fetch aborts
    LDR    pc, VECTOR_DAbort   ;// 10 - Data access aborts
    LDR    pc, VECTOR_Unused   ;// 14 - Reserved (was address exception)
    LDR    pc, VECTOR_IRQ      ;// 18 - IRQ interrupts
    LDR    pc, VECTOR_FIQ      ;// 1C - FIQ interrupts

;//Fill in the remaining addresses as appropriate
VECTOR_Reset DCD      xxxxxxxxxxxx ;// 00 - Reset
VECTOR_Undef DCD      xxxxxxxxxxxx ;// 04 - undef
VECTOR_SWI   DCD      xxxxxxxxxxxx ;// 08 - software interrupt
VECTOR_PAbort DCD      xxxxxxxxxxxx ;// 0C - prefetch abort
VECTOR_DAbort DCD      xxxxxxxxxxxx ;// 10 - data abort
VECTOR_Unused DCD      xxxxxxxxxxxx ;// 14 - reserved
VECTOR_IRQ   DCD      apINT_IRQDispatcher ;// 18 - IRQ
VECTOR_FIQ   DCD      apINT_FIQDispatcher ;// 1C - FIQ
```

12.5 Configuring the Interrupt Controller

The interrupt controller currently used (VIC or INT) is configured through the constants in `apintcfg.h`, which is in directory `apos`.

Note that the format of this file is unusual, as it is included by both assembler and C files. The main difference is that the preprocessor instruction `# define` must be written with a space after the `#` character.

Note that the object-only code release is pre-configured as described below.

12.5.1 `apINT_VERSION`

This specifies the version of the interrupt controller and is one of the options below:

«Note» In the object code build, the `apVERSION_ARMULATOR` option is not available.

12.5.1.1 `apVERSION_INTEGRATOR`

This assumes an RPS interrupt controller of the type present on an Integrator development platform.

If running on an Integrator platform this assumes an RPS controller in the Integrator AP, with an optional logic module. If present, the logic module also includes an RPS controller.

12.5.1.2 `apVERSION_ARMULATOR`

ARMulator model, assuming a single RPS controller.

12.5.1.3 `apVERSION_VECTORED`

A single PrimeCell vectored interrupt controller PL190

12.5.1.4 `apVERSION_VIC_ON_LM`

Integrator platform - this assumes an RPS controller in the Integrator AP, with an optional logic module. If present, the logic module includes a PrimeCell vectored PL190 controller.

12.5.1.5 `apVERSION_NONE_ON_LM`

Integrator platform - this assumes an RPS controller in the Integrator AP, with an optional logic module. If present, the logic module includes no interrupt controller.

12.5.2 `apCONFIG_CORE_SUPPORTS`

This must be defined as one of the following:

- ♦ `apCONFIG_ARM_AND_THUMB` indicates that the core can support both ARM and Thumb states and the assembler code is then compiled for interworking.
- ♦ `apCONFIG_ARM_ONLY` indicates that the core only supports ARM code (e.g. StrongARM).

«Note» In the object code build, this has been defined as `apCONFIG_ARM_AND_THUMB`.

12.5.3 `apOS_CONFIG_INT_REENTRANT`

This exists for expansion only and must be defined as `FALSE`.

«Note» In the object code build, this has been defined as `FALSE`.

12.5.4 apOS_CONFIG_INT_USE_PREDISPATCH_CODE

- ◆ if defined as TRUE, the Interrupt controller executes a specified routine before calling each Interrupt Service Routine
- ◆ if defined as FALSE, no pre-dispatch routine is available, but there is a slight speed gain.

This routine is specified using apINT_IRQPreDispatchSet.

«Note» In the object code build, this has been defined as FALSE.

12.5.5 apOS_CONFIG_INT_USE_POSTDISPATCH_CODE

- ◆ if defined as TRUE, the Interrupt controller executes a specified routine after calling each Interrupt Service Routine
- ◆ if defined as FALSE, no pre-dispatch routine is available, but there is a slight speed gain.

This routine is specified using apINT_IRQPostDispatchSet.

«Note» In the object code build, this has been defined as FALSE.

12.5.6 apOS_CONFIG_INT_ISR_ISA

- ◆ If defined as apCONFIG_ARM_ONLY, interrupt service routines are expected to be in ARM code. Veneers are added by the linker when interworking with Thumb ISRs as usual.
- ◆ If defined as apCONFIG_THUMB_ONLY, interrupt service routines are expected to be in Thumb code. Veneers are added by the linker when interworking with ARM ISRs as usual.

«Note» In the object code build, this has been defined as apCONFIG_ARM_ONLY.

12.5.7 apOS_INT_CONFIG_SOURCES_USED

Each interrupt controller can control up to 32 interrupt sources. By default, enough space is assigned for this many sources. However, if a lesser number of sources are required, this can be specified by defining the constant apOS_INT_CONFIG_SOURCES_USED. This constant must specify the total number of interrupt sources used across all interrupt controllers as below

```
#define apOS_INT_CONFIG_SOURCES_USED 16
```

«Note» In the object code build, this has been defined as 16.

12.5.8 apOS_CONFIG_VIC_AT_0xFFFFF000

In a real system, a Vectored Interrupt Controller (if present) must be placed at 0xFFFFF000 in the memory map, although this might not be possible in a development system.

This means that a dispatcher need not be called for interrupts, so saving a number of cycles. See the PL190 documentation.

This is indicated by setting this constant as below:

```
# define apOS_CONFIG_VIC_AT_0xFFFFF000 1
```

«Note» This relates only to the Vectored Controller (PL190) and so has no effect in the object build.

12.6 Developing on the Integrator Platform

When developing on the Integrator platform, it might be desirable to have a PL190 Vectored Interrupt Controller (VIC) on the Logic Module in addition to the standard Interrupt Controller on the main AP motherboard.

To configure the system to support this, specify in `apintcfg.h`:

```
# define apINT_VERSION apVERSION_VIC_ON_LM
```

This modifies the default behaviour of the system as follows:

- ◆ Interrupt controllers up to the number specified in `apOS_INT_MAXIMUM` are standard RPS controllers. By default, this is a single controller.
- ◆ Interrupt controllers beyond this number are VIC controllers.
- ◆ The indirection layer `os.c` routes function calls to the correct module. For example, under the default configuration, the call `apOS_INT_InterruptEnable(0x101)` is decoded as:
 - interrupt source 0x101 decodes as Controller 1, Interrupt Source 1
 - controller 1 is a VIC (Controller 0 is the RPS controller)
 - so the required call is to VIC instance 0 `apVIC_InterruptEnable(0x001)`
- ◆ The standard Interrupt Controller always uses the IRQ vector and the VIC uses the FIQ vector.

«Note» This option is available if using the object build of the RPS interrupt controller with the source of the vectored interrupt controller.

12.7 Restrictions and features

The software for handling interrupts is not re-entrant. All interrupt handling code is run in IRQ mode, with interrupts to the processor core disabled.

Any code that you write to handle interrupts must not re-enable interrupts to the core.

In particular, if you change processor mode, care must be taken not to re-enable interrupts to the core.

The function `apINT_SenseSet` is implemented. However, because it can only set the default reset sense, with which all interrupt sources are already initialized, this function is of no real use.